

Gateway API

- 🌐 Gateway API is the next-generation networking API in Kubernetes that improves the traditional Ingress resource.
- 🌐 It provides advanced traffic management, supports HTTP , HTTPS , TCP , UDP , and gRPC traffic , and separates infrastructure management from application routing.
- 🏛️ The main resources are GatewayClass , Gateway , and HTTPRoute . Traffic flows from Gateway to Route and then to Services and Pods.
- 💡 It is commonly used for modern traffic routing , canary deployments , and multi-application environments ..
- ⚡ In Kubernetes Gateway API, traffic flows like this:



Earlier, Kubernetes used Ingress to manage incoming traffic. But Ingress has limitations:

- 🌐 Only HTTP/HTTPS
 - Complex configurations
 - Different controllers behave differently
 - 🏛️ Therefore Kubernetes introduced Gateway API .
-

⚡ Before using Gateway API, you must install the Gateway API CRDs.

Step 1: 🛑 Install Gateway API CRDs

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.0.0/gateway-api.yaml
```

🚀 verify

```
kubectl get crd | grep gateway
kubectl get gatewayclass
```

⚡ Install a Gateway Controller

- Gateway API CRDs alone are not enough.
- You need a controller such as:
 - 🛑 Istio
 - 🛑 Envoy Proxy
 - 🛑 HAProxy Technologies
- 🚀 The controller watches Gateway resources and creates load balancers/proxies.

Step 2: ⚡ Install Istio

```
curl -L https://istio.io/downloadIstio | sh -
cd istio-*
export PATH=$PWD/bin:$PATH
istioctl version
```

Step 3: 🌐 Install Istio Control Plane

```
istioctl install --set profile=demo -y
kubectl get pods -n istio-system
```

🚀 Components of Gateway API

- There are 3 main objects. ### 1. GatewayClass

- First Kubernetes needs to know Who will manage the Gateway?
- 💡 Just like StorageClass tells Kubernetes which storage driver to use.
- ⚡ Example:

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: istio
spec:
  controllerName: istio.io/gateway-controller # Use NGINX Controller
```

2. Gateway

- 🏛 Gateway is the entry point into the cluster.
- 🌐 It listens on specific ports and protocols (HTTP, HTTPS, TCP, gRPC, etc.) and forwards traffic to Routes such as HTTPRoute, which then direct traffic to Services and Pods .
- Client → Gateway → Route → Service → Pods
- ⚡ Example:

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: nginx-gateway
spec:
  gatewayClassName: istio
  listeners:
  - name: https
    protocol: HTTPS
    port: 443 # Listen on Port 443 (Accept HTTPs traffic)
    hostname: example.com # Accepts traffic for example.com
    tls:
      mode: Terminate
      certificateRefs:
      - kind: Secret
        name: tls-secret
```

3. 🌐 HTTPRoute

- 🏛 Now Gateway receives traffic

- ⚡ Where should it send traffic? HTTPRoute decides.
- 🌐 Gateway API does not have an HTTPSRoute resource . HTTPS traffic is handled using an HTTPS listener on a Gateway and routed using HTTPRoute .
- Example:

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app-route
spec:
  parentRefs:
    - name: nginx-gateway
  hostnames:
    - example.com
  rules:
    - matches:
      - path:
          type: PathPrefix
          value: /app # If URL starts with /app -- send traffic t
    backendRefs:
      - name: app-service
        port: 80
```

Available Route Types in Gateway API

 Route Type	 Purpose	 Traffic Type	 Common Use Case
 HTTPRoute	HTTP/HTTPS routing	Layer 7 (HTTP/HTTPS)	Web applications, REST APIs
 GRPCRoute	gRPC routing	Layer 7 (gRPC)	Microservices using gRPC
 TCPRoute	TCP traffic routing	Layer 4 (TCP)	MySQL, PostgreSQL, Redis
 TLSRoute	TLS passthrough routing	Encrypted TLS traffic	End-to-end TLS applications
 UDPRoute	UDP traffic routing	Layer 4 (UDP)	DNS, gaming, streaming services

🌐 Kubernetes Gateway API Components

 Component	 Purpose	 Description	 Example
 GatewayClass	Defines gateway implementation	Specifies which controller manages Gateways technology?	NGINX, Istio, AWS Gateway API Controller, haproxy
 Gateway	Entry point for traffic	Defines listeners, ports, protocols, and hostnames	HTTP/HTTPS traffic on port 80/443
 HTTPRoute	HTTP routing rules	Routes HTTP requests based on hostnames, paths, headers	"Where should HTTP traffic go?" /api → backend service
 TCPRoute	TCP traffic routing	"Where should TCP traffic go?" Routes Layer 4 TCP traffic	MySQL, PostgreSQL traffic
 GRPCRoute	gRPC traffic routing	"Where should gRPC traffic go?" Routes gRPC requests	Microservices using gRPC
 Service	Stable endpoint for Pods	Provides a consistent network endpoint for Pods	frontend-service
 Pod	Runs the application	Smallest deployable Kubernetes unit	Nginx, Java, Node.js application

-  Gateway API is the modern replacement for Ingress that provides advanced traffic management using GatewayClass, Gateway, and Route resources.

 If you have 3 microservices (frontend, user-service, order-service), a single Gateway can route traffic to all three services using one or more HTTPRoute rules.

single HTTPRoute:

 Request Flow

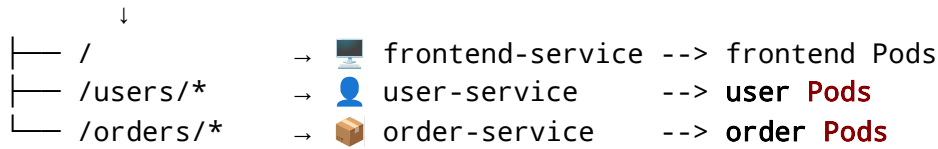
 User Request



 Gateway



 HTTPRoute



1. Gateway

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: microservices-gateway
spec:
  gatewayClassName: istio
  listeners:
    - name: http
      protocol: HTTP
      port: 80
```

2. HTTPRoute

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app-routes
spec:
  parentRefs:
    - name: microservices-gateway

  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
        backendRefs:
          - name: frontend-service
            port: 80

    - matches:
        - path:
            type: PathPrefix
            value: /users
        backendRefs:
          - name: user-service
            port: 80
```

- matches:
 - path:
 - type: PathPrefix
 - value: /orders
- backendRefs:
 - name: order-service
 - port: 80

3. Services

Frontend Service

```
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  selector:
    app: frontend
  ports:
    - port: 80
```

User Service

```
apiVersion: v1
kind: Service
metadata:
  name: user-service
spec:
  selector:
    app: users
  ports:
    - port: 80
```

Order Service

```
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: orders
```

ports:
- port: 80

How does Gateway API work with multiple microservices?

- 🌐 A single Gateway receives incoming traffic.
- 🚀 An HTTPRoute uses path-based or host-based routing rules to forward requests to different Kubernetes Services.

🌐 Gateway API / HTTPRoute Path-Based Routing

🌐 URL Request	🎯 Destination Service	📖 Routing Rule	💡 Purpose
http://app.com/	💻 frontend-service	Match root path /	Serve frontend UI
http://app.com/users	👤 user-service	Match /users prefix	User management APIs
http://app.com/users/profile	👤 user-service	Match /users/* prefix	User profile operations
http://app.com/orders	📦 order-service	Match /orders prefix	Order management APIs
http://app.com/orders/123	📦 order-service	Match /orders/* prefix	Specific order details

🌐 Gateway API / HTTPRoute Host-Based Routing

🚀 Request Flow



└─ user.example.com → 👤 user-service
└─ order.example.com → 📦 order-service

 Hostname	 Destination Service	 Routing Rule	 Purpose
app.example.com	 frontend-service	Match hostname app.example.com	Serve frontend application
user.example.com	 user-service	Match hostname user.example.com	User management APIs
order.example.com	 order-service	Match hostname order.example.com	Order management APIs

Example HTTPRoute

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app-route
spec:
  parentRefs:
    - name: app-gateway

  hostnames:
    - "app.example.com"

  rules:
    - backendRefs:
        - name: frontend-service
          port: 80
```

Gateway API / Istio Routing Types :

 Routing Type	 How Routing Works	 Example	 Use Case
 Host-Based Routing	Routes traffic based on hostname/domain	app.example.com → Service A	Multiple applications on different domains/subdomains

 Routing Type	 How Routing Works	 Example	 Use Case
 Path-Based Routing	Routes traffic based on URL path	/api → Service A /orders → Service B	Microservices under a single domain
 Header-Based Routing	Routes traffic based on HTTP headers	version=v2 → Service B	Canary deployments, A/B testing, user segmentation

Kubernetes Gateway API – Rapid Fire Interview Questions & Answers

 Question	 Answer
♦ What is Gateway API?	Gateway API is the next-generation Kubernetes traffic management API that provides routing, load balancing, and traffic control.
♦ Why was Gateway API introduced?	To overcome Ingress limitations and provide more flexible, role-oriented traffic management.
♦ What replaces Ingress in Gateway API?	Gateway + Routes (HTTPRoute , TCPRoute , GRPCRoute , etc.).
♦ What is GatewayClass?	Defines which controller manages Gateways .
♦ Is GatewayClass Namespaced?	 No, it is Cluster-scoped.
♦ What is Gateway?	Entry point for external traffic into the cluster.
♦ What is HTTPRoute?	Defines HTTP routing rules .
♦ What is TCPRoute?	Routes TCP traffic.
♦ What is UDPRoute?	Routes UDP traffic.
♦ What is TLSRoute?	Routes encrypted TLS traffic without terminating TLS .
♦ What is GRPCRoute?	Routes gRPC traffic.
♦ Is HTTPSRoute available?	 No. Use HTTPS listener in Gateway with HTTPRoute .
♦ What is parentRefs?	Connects a Route to a Gateway .
♦ What is backendRefs?	Defines the backend Service receiving traffic .

 Question	 Answer
♦ What is hostname routing?	Routing based on domain names . <code>user.example.com</code>
♦ What is path-based routing?	Routing based on URL paths . <code>/user</code>
♦ Can one Gateway have multiple Routes?	 Yes.
♦ Can one Route point to multiple Services?	 Yes (traffic splitting).
♦ Does Gateway API support traffic splitting?	 Yes.
♦ Does Gateway API support canary deployments?	 Yes.
♦ Does Gateway API support blue-green deployments?	 Yes.
♦ Does Gateway API support TLS termination?	 Yes.
♦ What port is typically used for HTTP?	80
♦ What port is typically used for HTTPS?	443
♦ Which resource performs TLS termination?	Gateway
♦ What is <code>allowedRoutes</code> ?	Controls which Routes can attach to a Gateway.
♦ Can Routes exist in different namespaces?	 Yes.
♦ What is cross-namespace routing?	Route and Gateway can exist in different namespaces.
♦ Which resource actually sends traffic to Pods?	Service
♦ Do Routes directly connect to Pods?	 No, they connect to Services.
♦ How does traffic flow in Gateway API?	Client → Gateway → Route → Service → Pod
♦ Who receives traffic first?	Gateway

 Question	 Answer
♦ Who makes routing decisions?	Route
♦ Who load balances traffic?	Service
♦ Where does the application run?	Pod

- 🚀 What TLS Termination ?
- 💡 TLS Termination means HTTPS traffic is decrypted at the Gateway , and then forwarded to the backend service as plain HTTP .
- 💡 TLS certificate stored in a Kubernetes Secret to decrypt the traffic, and then forwards the request to an HTTPRoute , Service , and Pods .
- Flow :

